

HistoShare

Extension Firefox — Rapport Technique

Version 1.0 · 11 mai 2026

Résumé du projet

HistoShare est une extension Firefox permettant le partage d'historique de navigation entre utilisateurs, enrichi d'un système de tags (mots-clés) pour organiser et filtrer les liens. Le projet s'appuie sur les APIs natives du navigateur et sur Firebase comme backend de synchronisation en temps réel.

Nom du projet	HistoShare
Type	Extension Firefox (WebExtension — Manifest V2)
Langages	HTML5, CSS3, JavaScript (ES6+)
Backend	Firebase Realtime Database (API REST)
Environnement	VS Code + Firefox (version 150+)
Statut	Fonctionnel — version de test local

KONÉ KPANTIERI ISMAËL

Auteur

DIABY ISHAK BAKARY

Auteur

*Sous la supervision de
Dr Diallo*

Table des matières

- | | |
|----|---|
| 1. | Contexte et objectifs |
| 2. | Architecture de l'extension |
| 3. | Structure des fichiers |
| 4. | Fonctionnalités implémentées |
| 5. | Guide d'installation et de test |
| 6. | Intégration Firebase — Partage entre utilisateurs |
| 7. | Extraits de code commentés |
| 8. | Pistes d'amélioration |
| 9. | Conclusion |

1. Contexte et objectifs

1.1 Contexte

Ce projet a été développé dans le cadre d'un apprentissage des extensions Firefox depuis zéro. L'objectif était de créer une extension fonctionnelle, documentée et testable immédiatement, en progressant par étapes successives.

1.2 Objectifs fonctionnels

- Lire et afficher l'historique de navigation de l'utilisateur
- Permettre l'ajout de tags (mots-clés) sur chaque lien visité
- Filtrer les liens par tag pour retrouver rapidement une page
- Partager l'historique enrichi entre plusieurs utilisateurs via un serveur distant

1.3 Contraintes techniques

Manifest	Version 2 (compatibilité Firefox stable)
Stockage local	browser.storage.local — données persistantes par navigateur
Stockage distant	Firebase Realtime Database via API REST (sans SDK)
Permissions	history, storage, tabs, host_permissions Firebase

2. Architecture de l'extension

L'extension suit l'architecture standard WebExtension de Mozilla. Elle se compose de trois couches :

Couche	Composants	Rôle
Interface	popup.html + popup.css	Affichage et interactions utilisateur
Logique	popup.js + sync.js	Traitement des données, appels APIs
Déclaration	manifest.json	Permissions, points d'entrée, métadonnées

2.1 Flux de données

Lors de l'ouverture du popup, le flux est le suivant :

- **popup.js** interroge **browser.history.search()** → liste des URLs visitées
- **browser.storage.local.get()** → tags associés à chaque URL
- Les deux sources sont fusionnées et affichées dans **popup.html**
- Lors d'un partage, **sync.js** envoie un POST JSON vers Firebase
- L'onglet 'Liens partagés' effectue un GET Firebase → affichage des liens communautaires

3. Structure des fichiers

Fichier	Rôle	Taille est.
manifest.json	Déclaration de l'extension : permissions, popup, version	~30 lig.
popup.html	Structure HTML de l'interface : onglets, filtre, liste	~40 lig.
popup.css	Styles visuels : mise en page, tags, boutons, badges	~90 lig.
popup.js	Logique principale : historique, tags, filtrage, partage	~160 lig.
sync.js	Communication Firebase : partagerLien(), recupererLiensPartages()	~60 lig.

Arborescence du projet :

```
mon-extension/  
manifest.json <-- point d'entrée obligatoire  
popup.html <-- interface utilisateur  
popup.css <-- styles  
popup.js <-- logique principale  
sync.js <-- module de synchronisation Firebase
```

4. Fonctionnalités implémentées

Fonctionnalité	Technologie	Statut
Affichage de l'historique (20 liens, 7 jours)	browser.history API	● Implémentée
Ajout de tags par lien (touche Entrée)	browser.storage.local	● Implémentée
Filtrage par tag en temps réel	Array.filter() JS	● Implémentée
Persistence des tags entre sessions	browser.storage.local	● Implémentée
Partage d'un lien vers Firebase	fetch() + REST API	● Implémentée
Affichage des liens partagés par d'autres	fetch() GET Firebase	● Implémentée
Identification anonyme de l'utilisateur	ID généré localement	● Implémentée
Filtrage des liens partagés par tag	Prévu — non implémenté	● En attente
Suppression d'un tag ou d'un lien partagé	Prévu — non implémenté	● En attente

5. Guide d'installation et de test

5.1 Chargement dans Firefox

- 1 Ouvrir Firefox et saisir `about : debugging` dans la barre d'adresse
- 2 Cliquer sur 'Ce Firefox' dans le menu gauche
- 3 Cliquer sur 'Charger un module complémentaire temporaire'
- 4 Naviguer jusqu'au dossier du projet et sélectionner `manifest.json`
- 5 L'extension apparaît dans la liste — cliquer sur l'icône dans la barre Firefox

5.2 Rechargement après modification

Après chaque modification de code, retourner sur `about : debugging` et cliquer **Recharger** à côté de l'extension. La console du popup est accessible via clic droit sur le popup → **Inspecter**.

5.3 Scénario de test recommandé

- Ouvrir le popup → vérifier que l'historique s'affiche (onglet 'Mon historique')
- Saisir un tag dans le champ '+ tag' d'un lien, puis appuyer sur Entrée
- Fermer et rouvrir le popup → le tag doit persister
- Utiliser le champ 'Filtrer par tag' → seuls les liens avec ce tag s'affichent
- Cliquer '■ Partager' sur un lien → le bouton doit indiquer '✓ Partagé !'
- Basculer sur l'onglet 'Liens partagés' → le lien apparaît

6. Intégration Firebase — Partage entre utilisateurs

6.1 Pourquoi un serveur est nécessaire

Le partage entre plusieurs utilisateurs implique un point de stockage central accessible par tous. Les extensions Firefox n'offrent pas de mécanisme peer-to-peer natif. Firebase Realtime Database a été retenu pour sa simplicité de mise en œuvre : aucun SDK à installer, l'API REST suffit.

6.2 Configuration Firebase (étapes)

- **Étape 1** : Créer un projet sur console.firebase.google.com
- **Étape 2** : Activer 'Realtime Database' → mode test (règles ouvertes)
- **Étape 3** : Copier l'URL de la base (ex: <https://mon-projet-rtdb.firebaseio.com>)
- **Étape 4** : Remplacer `FIREBASE_URL` dans `sync.js` par cette URL
- **Étape 5** : Ajouter l'URL dans `host_permissions` du `manifest.json`
- **Étape 6** : Recharger l'extension dans `about:debugging`

6.3 Structure des données dans Firebase

```
liens_partages/  
-NxKj2abc123/  
url: 'https://example.com/article'  
titre: 'Titre de la page'  
tags: ['dev', 'firefox']  
partageA: 1716300000000 // timestamp Unix  
utilisateur: 'user_a7f3k'
```

6.4 Comparatif des solutions de backend

Solution	Avantages	Inconvénients	Coût
Firebase (choix actuel)	Simple, temps réel, bien documenté	Dépendance Google	Gratuit (limites)
Supabase	Open source, PostgreSQL	Légèrement plus complexe	Gratuit
JSONBin.io	Ultra simple, REST pur	Pas de temps réel	Gratuit

7. Extraits de code commentés

7.1 Lecture de l'historique (popup.js)

```
browser.history.search({
  text: '', // chaîne vide = toutes les pages
  startTime: Date.now() - 7 * 24 * 60 * 60 * 1000, // 7 jours
  maxResults: 20 // limite à 20 résultats
}).then(function(items) {
  // items : tableau d'objets { url, title, lastVisitTime }
  afficherLiens(items);
});
```

7.2 Sauvegarde d'un tag (browser.storage)

```
browser.storage.local.get('tagsParUrl').then(function(resultat) {
  const tagsParUrl = resultat.tagsParUrl || {};
  const tags = tagsParUrl[url] || [];
  if (!tags.includes(nouveauTag)) {
    tags.push(nouveauTag);
    tagsParUrl[url] = tags;
    browser.storage.local.set({ tagsParUrl }); // persistance
  }
});
```

7.3 Envoi vers Firebase (sync.js)

```
function partagerLien(url, titre, tags) {
  const lien = { url, titre, tags, partageA: Date.now() };
  return fetch(FIREBASE_URL + '/liens_partages.json', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(lien)
  });
}
```

8. Pistes d'amélioration

8.1 Court terme

- Filtrage des liens partagés par tag (côté client)
- Suppression d'un tag ou d'un lien partagé
- Affichage de l'URL complète au survol
- Pagination ou chargement infini pour les longs historiques

8.2 Moyen terme

- Authentification Firebase (email/anonyme) pour identifier les utilisateurs
- Règles de sécurité Firebase (lecture/écriture selon authentification)
- Export des liens et tags en JSON ou CSV
- Recherche plein texte dans les titres et URLs

8.3 Long terme

- Migration vers Manifest V3 (futur standard Firefox et Chrome)
- Publication sur addons.mozilla.org (AMO)
- Groupes / collections partagées entre utilisateurs sélectionnés
- Suggestions automatiques de tags par analyse du contenu de la page

9. Conclusion

L'extension HistoShare démontre qu'il est possible de créer, sans connaissance préalable des extensions Firefox, un outil fonctionnel couvrant :

- L'accès aux APIs sensibles du navigateur (historique, stockage)
- Une interface utilisateur intégrée via le système de popup
- La persistance locale de données structurées
- La communication avec un service cloud externe via fetch()

Le projet constitue une base solide pour une extension plus complète. Les prochaines étapes prioritaires sont la configuration Firebase avec une vraie URL de projet, puis l'ajout de règles de sécurité pour protéger les données partagées.

Document généré le 11 mai 2026 — Projet HistoShare v1.0